Status	Finished
Started	Friday, 30 October 2026, 6:37 PM
Completed	Friday, 30 October 2026, 6:42 PM
Duration	4 mins 52 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[i] - A[j] = k, i != j.

Input Format

1. First line is number of test cases T. Following T lines contain:
2. N, followed by N integers of the array
3. The non-negative integer k

Output format

Print 1 if such a pair exists and 0 if it doesn't.

Example

Input

1

3

1

3

5

4

Output:

1

Input

1

3

1

3

5

99

Output

0

For example:

Input	Result
1 3 1 3 5 4	1
1 3 1 3 5 99	0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return
```

```
    iterator = iter(input_data)

    # Read number of test cases
    try:
        T = int(next(iterator))
    except StopIteration:
        return

    for _ in range(T):
        N = int(next(iterator))
        A = [int(next(iterator)) for _ in range(N)]
        k = int(next(iterator))

        # Two-pointer logic
        i = 0
        j = 1
        found = 0

        while i < N and j < N:
            if i != j and A[j] - A[i] == k:
                found = 1
                break
            elif A[j] - A[i] < k:
                j += 1
            else:
                i += 1

            if i == j:
                j += 1

        print(found)

if __name__ == '__main__':
    solve()

```

	Input	Expected	Got	
✓	1 3 1 3 5 4	1	1	✓
✓	1 3 1 3 5 99	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

An array is monotonic if it is either **monotone increasing** or **monotone decreasing**.
An array A is monotone increasing if for all i <= j, **A[i] <= A[j]**. An array A is monotone decreasing if for all i <= j, **A[i] >= A[j]**.

Write a program if n array is monotonic or not. Print "True" if is monotonic or "False" if it is not. Array can be monotone increasing or decreasing.

Input Format:

First line n-get number of elements

Next n Lines is the array of elements

Output Format:

True ,if array is monotone increasing or decreasing.

otherwise False is printed

Sample Input1

4

5

6

7

8

Sample Output1

True

Sample Input2

4

6

5

4

3

Sample Output2

True

Sample Input 3

4

6

7

8

7

Sample Output3

False

For example:

Input	Result
4 6 5 4 3	True

Answer: (penalty regime: 0 %)

```
import sys

def is_monotonic():
    # Read all lines from standard input
```

```
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Parse the number of elements
    n = int(input_data[0])

    # Parse the array elements
    A = [int(x) for x in input_data[1:n+1]]

    # Track if the array is increasing or decreasing
    increasing = True
    decreasing = True

    # Check adjacent elements
    for i in range(len(A) - 1):
        if A[i] > A[i + 1]:
            increasing = False
        if A[i] < A[i + 1]:
            decreasing = False

    # If it satisfies either condition, it is monotonic
    if increasing or decreasing:
        print("True")
    else:
        print("False")

if __name__ == '__main__':
    is_monotonic()
```

	Input	Expected	Got	
✓	4 6 5 4 3	True	True	✓
✓	4 3 5 7 4	False	False	✓
✓	4 1 6 9 2	False	False	✓
✓	4 9 6 4 2	True	True	✓
✓	3 2 1 4	False	False	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1

2

2

3

4

Output:

1 2 3 4

Example Input:

6

1

1

2

2

3

3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4
6 1 1 2 2 3 3	1 2 3

Answer: (penalty regime: 0 %)

```
import sys

def print_distinct_elements():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
```

```
        return

    # Extract the length of the array
    n = int(input_data[0])

    # Extract the n array elements
    A = [int(x) for x in input_data[1:n+1]]

    # Track seen elements to maintain unique status and preserve insertion order
    seen = set()
    distinct_elements = []

    for num in A:
        if num not in seen:
            seen.add(num)
            distinct_elements.append(str(num))

    # Print elements in a single line, space-separated
    print(" ".join(distinct_elements))

if __name__ == '__main__':
    print_distinct_elements()

```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a matrix mat where every row is sorted in **strictly increasing** order, return the **smallest common element** in all rows.

If there is no common element, return -1.

Example 1:

Input:

4 5

1 2 3 4 5

2 4 5 8 10

3 5 7 9 11

1 3 5 7 9

Output:

5

Constraints:

- 1 <= mat.length, mat[i].length <= 500
- 1 <= mat[i][j] <= 10^4
- mat[i] is sorted in strictly increasing order.

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def smallest_common_element():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    try:
        # Read matrix dimensions
        R = int(next(iterator))
        C = int(next(iterator))

        counts = Counter()

        # Parse the matrix row by row
        for _ in range(R):
            for _ in range(C):
                val = int(next(iterator))
                counts[val] += 1

        # Since rows are sorted, the first element with frequency == R
        # when scanning sorted unique keys will be the smallest common element
        for val in sorted(counts.keys()):
            if counts[val] == R:
                print(val)
                return

        # If no common element is found
        print(-1)
```

```
34
35      except StopIteration:
36          print(-1)
37
38  if __name__ == '__main__':
39      smallest_common_element()
40
```

	Input	Expected	Got	
✓	4 5 1 2 3 4 5 2 4 5 8 10 3 5 7 9 11 1 3 5 7 9	5	5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given two arrays of positive integers, for each element in the second array, find the total number of elements in the first array which are *less than or equal to* that element. Store the values determined in an array.

For example, if the first array is *[1, 2, 3]* and the second array is *[2, 4]*, then there are *2* elements in the first array *less than or equal to 2*. There are *3* elements in the first array which are *less than or equal to 4*. We can store these answers in an array, *answer = [2, 3]*.

Program Description

The program must return an array of *m* positive integers, one *for each maxes[i]* representing the total number of elements *nums[j]* satisfying *nums[j] ≤ maxes[i]* where *0 ≤ j < n* and *0 ≤ i < m,* in the given order.

The program has the following:

nums[nums[0],...nums[n-1]]: first array of positive integers
maxes[maxes[0],...maxes[n-1]]: second array of positive integers

Constraints

- *2 ≤ n, m ≤ 10^5*
- *1 ≤ nums[j] ≤ 10^9*, where *0 ≤ j < n*.
- *1 ≤ maxes[i] ≤ 10^9*, where *0 ≤ i < m*.

Input Format For Custom Testing

Input from stdin will be processed as follows and passed to the program.

The first line contains an integer *n*, the number of elements in *nums*.
The next *n* lines each contain an integer describing *nums[j]* where *0 ≤ j < n.*
The next line contains an integer *m*, the number of elements in *maxes*.
The next *m* lines each contain an integer describing *maxes[i]* where *0 ≤ i < m.*

Sample Case 0

Sample Input 0

```
4
1
4
2
4
2
3
5
```

Sample Output 0

```
2
4
```

Explanation 0

We are given *n = 4*, *nums = [1, 4, 2, 4]*, *m = 2*, and *maxes = [3, 5]*.

1. For *maxes[0] = 3*, we have *2* elements in *nums* (*nums[0] = 1* and *nums[2] = 2*) that are *≤ maxes[0]*.
2. For *maxes[1] = 5*, we have *4* elements in *nums* (*nums[0] = 1*, *nums[1] = 4*, *nums[2] = 2*, and *nums[3] = 4*) that are *≤ maxes[1]*.

Thus, the program returns the array *[2, 4]* as the answer.

Sample Case 1

Sample Input 1

```
5
2
10
5
4
8
4
3
1
7
8
```

Sample Output 1

```
1
0
3
4
```

Explanation 1

We are given, *n = 5*, *nums = [2, 10, 5, 4, 8]*, *m = 4*, and *maxes = [3, 1, 7, 8]*.

1. For *maxes[0] = 3*, we have *1* element in *nums* (*nums[0] = 2*) that is ≤ *maxes[0]*.
2. For *maxes[1] = 1*, there are *0* elements in *nums* that are ≤ *maxes[1]*.
3. For *maxes[2] = 7*, we have *3* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, and *nums[3] = 4*) that are ≤ *maxes[2]*.
4. For *maxes[3] = 8*, we have *4* elements in *nums* (*nums[0] = 2*, *nums[2] = 5*, *nums[3] = 4*, and *nums[4] = 8*) that are ≤ *maxes[3]*.

Thus, the program returns the array *[1, 0, 3, 4]* as the answer.

Answer: (penalty regime: 0 %)

```python
import sys
import bisect

def counts_less_than_or_equal():
    # Read all tokens from standard input efficiently
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    try:
        # Read elements for the 'nums' array
        n = int(next(iterator))
        nums = [int(next(iterator)) for _ in range(n)]

        # Read elements for the 'maxes' array
        m = int(next(iterator))
        maxes = [int(next(iterator)) for _ in range(m)]

        # Step 1: Sort the nums array
        nums.sort()

        # Step 2: Use binary search for each maxes element
        for x in maxes:
            # bisect_right finds the index of the first element strictly greater than x
            # which is exactly the count of elements less than or equal to x
            count = bisect.bisect_right(nums, x)
            print(count)

    except StopIteration:
        return

if __name__ == '__main__':
    counts_less_than_or_equal()

```

	Input	Expected	Got	
✓	4 1 4 2 4 2 3 5	2 4	2 4	✓
✓	5 2 10 5 4 8 4 3 1 7 8	1 0 3 4	1 0 3 4	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the pth element of the list, sorted ascending. If there is no p^{th} element, return 0.

Example

n = 20

p = 3

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if p = 3, then 4 is returned. If p > 6, 0 would be returned.

Constraints

$1 \leq n \leq 10^{15}$

$1 \leq p \leq 10^{9}$

The first line contains an integer n, the number to factor.

The second line contains an integer p, the 1-based index of the factor to return.

Sample Case 0

Sample Input 0

10

3

Sample Output 0

5

Explanation 0

Factoring n = 10 results in {1, 2, 5, 10}. Return the p = 3rd factor, 5, as the answer.

Sample Case 1

Sample Input 1

10

5

Sample Output 1

0

Explanation 1

Factoring n = 10 results in {1, 2, 5, 10}. There are only 4 factors and p = 5, therefore 0 is returned as the answer.

Sample Case 2

Sample Input 2

1

1

Sample Output 2

1

Explanation 2

Factoring n = 1 results in {1}. The p = 1st factor of 1 is returned as the answer.

For example:

Input	Result
10 3	5
10 5	0

Input	Result
1 1	1

Answer: (penalty regime: 0 %)

```
import sys
import math

def kth_factor():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    p = int(input_data[1])

    # Lists to store factors
    small_factors = []
    large_factors = []

    # Iterate up to the square root of n
    limit = int(math.isqrt(n))
    for i in range(1, limit + 1):
        if n % i == 0:
            small_factors.append(i)
            # Ensure we do not add the square root twice for perfect squares
            if i * i != n:
                large_factors.append(n // i)

    # large_factors are collected in descending order, so reverse them
    large_factors.reverse()

    # Combine both parts to get the full sorted factors list
    all_factors = small_factors + large_factors

    # Check if the p-th element (1-based index) exists
    if p <= len(all_factors):
        print(all_factors[p - 1])
    else:
        print(0)

if __name__ == '__main__':
    kth_factor()

```

	Input	Expected	Got	
✓	10 3	5	5	✓
✓	10 5	0	0	✓
✓	1 1	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Complete the program to count frequency of each element of an array. Frequency of a particular element will be printed once.

Sample Test Cases

Test Case 1

Input

7
23
45
23
56
45
23
40

Output

23 occurs 3 times
45 occurs 2 times
56 occurs 1 times
40 occurs 1 times

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def count_element_frequency():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First line contains the number of elements
    n = int(input_data[0])

    # Extract the n array elements
    A = [int(x) for x in input_data[1:n+1]]

    # Calculate the frequency of each element
    frequencies = Counter(A)

    # Track printed elements to ensure each unique element is printed only once
    seen = set()

    # Print the frequencies in the order they first appeared in the array
    for num in A:
        if num not in seen:
            seen.add(num)
            print(f"{num} occurs {frequencies[num]} times")

if __name__ == '__main__':
    count_element_frequency()

```

	Input	Expected	Got	
✓	7 23 45 23 56 45 23 40	23 occurs 3 times 45 occurs 2 times 56 occurs 1 times 40 occurs 1 times	23 occurs 3 times 45 occurs 2 times 56 occurs 1 times 40 occurs 1 times	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

The program must accept **N** integers and an integer **K** as the input. The program must print every K integers in descending order as the output.

Note: If **N % K != 0**, then sort the final N%K integers in descending order.

Boundary Condition(s):

1 <= N <= 10^4
-99999 <= Array Element Value <= 99999

Input Format:

The first line contains the values of N and K separated by a space.
The second line contains N integers separated by space(s).

Output Format:

The first line contains N integers.

Example Input/Output 1:

Input:

7 3
48 541 23 68 13 41 6

Output:

541 48 23 68 41 13 6

Explanation:

The first three integers are 48 541 23, after sorting in descending order the integers are **541 48 23**.
The second three integers are 68 13 41, after sorting in descending order the integers are **68 41 13**.
The last integer is **6**.
The integers are **541 48 23 68 41 13 6**
Hence the output is **541 48 23 68 41 13 6**.

Answer: (penalty regime: 0 %)

```
import sys

def sort_chunks_descending():
    # Read all input from standard input efficiently
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    try:
        # Read N and K
        N = int(next(iterator))
        K = int(next(iterator))
```

```
15
16          # Read all N integers
17          A = [int(next(iterator)) for _ in range(N)]
18
19          result = []
20
21          # Process the array in chunks of size K
22          for i in range(0, N, K):
23              # Extract the current chunk (automatically handles the remainder chunk if N % K != 0)
24              chunk = A[i:i+K]
25              # Sort the chunk in descending order
26              chunk.sort(reverse=True)
27              # Add to the final result list
28              result.extend(chunk)
29
30          # Print all integers space-separated on a single line
31          print(*(str(x) for x in result))
32
33      except StopIteration:
34          return
35
36  if __name__ == '__main__':
37      sort_chunks_descending()
38
```

	Input	Expected	Got	
✓	7 3 48 541 23 68 13 41 6	541 48 23 68 41 13 6	541 48 23 68 41 13 6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Assume you have an array of length ***n*** initialized with all **0**'s and are given ***k*** update operations.

Each operation is represented as a triplet: **[startIndex, endIndex, inc]** which increments each element of subarray **A[startIndex ... endIndex]** (startIndex and endIndex inclusive) with **inc**.

Return the modified array after all ***k*** operations were executed.

Example:

Input:

5

3

1 3 2

2 4 3

0 2 -2

Output:

-2 0 3 5 3

Explanation:

Initial state:

length = 5, updates = [[1,3,2],[2,4,3],[0,2,-2]]

[0,0,0,0,0]

After applying operation [1,3,2]:

[0,2,2,2,0]

After applying operation [2,4,3]:

[0,2,5,5,3]

After applying operation [0,2,-2]:

[-2,0,3,5,3]

Answer: (penalty regime: 0 %)

```
import sys

def range_addition():
    # Read all tokens from standard input efficiently
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    iterator = iter(input_data)

    try:
        # Read the array length (n) and number of operations (k)
        n = int(next(iterator))
        k = int(next(iterator))

        # Initialize the difference array with an extra buffer element
        diff = [0] * (n + 1)

        # Apply the difference rules for each update
        for _ in range(k):
```

```
        for _ in range(k):
            start = int(next(iterator))
            end = int(next(iterator))
            inc = int(next(iterator))

            diff[start] += inc
            if end + 1 < n:
                diff[end + 1] -= inc

        # Reconstruct the original array elements using prefix sum
        result = []
        current_sum = 0
        for i in range(n):
            current_sum += diff[i]
            result.append(str(current_sum))

        # Print the final array space-separated on a single line
        print(" ".join(result))

    except StopIteration:
        return

if __name__ == '__main__':
    range_addition()
```

	Input	Expected	Got	
✓	5 3 1 3 2 2 4 3 0 2 -2	-2 0 3 5 3	-2 0 3 5 3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an integer n, return an list of length n + 1 such that for each i (0 <= i <= n), ans[i] is the number of 1's in the binary representation of i.

Example:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101
```

Note: Complete the given function alone

For example:

Test	Result
print(CountingBits(5))	[0, 1, 1, 2, 1, 2]

Answer: (penalty regime: 0 %)

Reset answer

```python
def CountingBits(n: int) -> list[int]:
    # Initialize the array of size n + 1 with 0s
    ans = [0] * (n + 1)

    # Compute the number of 1s using the dynamic programming relation
    for i in range(1, n + 1):
        ans[i] = ans[i >> 1] + (i & 1)

    return ans

```

	Test	Expected	Got	
✓	print(CountingBits(2))	[0, 1, 1]	[0, 1, 1]	✓
✓	print(CountingBits(5))	[0, 1, 1, 2, 1, 2]	[0, 1, 1, 2, 1, 2]	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.